OS U4 Orange Problem

Name: Rekha Dhorigol

SRN: PES1UG24CS370

Section: G

Phase 5 & 6: Analysis-Only Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans: To implement pes checkout , three things must happen. First, .pes/HEAD is updated to contain ref: refs/heads/. Second, the target branch file (.pes/refs/heads/) is read to get the commit hash, and that commit's tree is loaded to get the full file snapshot. Third, the working directory is updated to match - files that exist in the new branch but not the current one are created, files that no longer exist are deleted, and files that differ are overwritten. This operation is complex because it must surgically modify the working directory without losing data. It needs to compare the current HEAD's tree against the target tree entry by entry, and apply only the necessary changes.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans: To detect a dirty working directory, compare each index entry's recorded mtime and size against the actual file on disk using stat(). If they differ, the file has been modified since it was last staged. Next, check if that same file differs between the current branch's tree and the target branch's tree by comparing their blob hashes. If both conditions are true —

the file is locally modified AND the branches disagree on its content — checkout must refuse to proceed, because continuing would silently destroy the user's unsaved changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans: "Detached HEAD" occurs when .pes/HEAD contains a raw commit hash instead of a branch reference like ref: refs/heads/main. Commits made in this state are written to the object store normally, but no branch pointer is updated to track them. This means once you switch away, those commits become unreachable through normal navigation — they exist on disk but nothing points to them. To recover, the user must remember the commit hash (visible in the terminal output after each commit). They can then run pes checkout -b new-branch to create a new branch pointing to that commit, making the chain reachable again. Without this, the commits will eventually be deleted by garbage collection.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans: The algorithm works in two phases: mark and sweep. In the mark phase, start from every branch in .pes/refs/heads/ and HEAD. For each branch, read the commit it points to, add its hash to a "reachable" set (implemented as a hash set for O(1) lookups), then follow the parent pointer to the previous commit, repeating until reaching the root commit. For each commit visited, also traverse its tree recursively — adding every tree and blob hash to the reachable set. In the sweep phase, list every file under .pes/objects/ and delete any whose hash is not in the reachable set. For a repository with 100,000 commits and 50 branches, assuming an average of 20 objects per commit (blobs + trees), you would visit roughly 2,000,000 objects in the mark phase. A hash set handles this efficiently in O(n) time.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans: Consider this race condition: a commit operation writes a new blob and tree to the object store, but has not yet called head_update() to move the branch pointer. At this exact moment, GC runs, scans all branches, does not see the new objects (nothing points to them yet), and deletes them. When the commit operation then tries to finalize by writing the commit object and updating HEAD, it now references objects that no longer exist — causing silent corruption. Git avoids this using a "grace period" strategy: GC never deletes objects newer than 2 weeks old, regardless of reachability. This gives any in-progress operations plenty of time to complete and make the objects reachable before GC could touch them. Git also uses lock files to prevent concurrent writes to the same refs.

Screenshots

Initial:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ export PES_AUTHOR="Rekha Dhorigol <PES1UG24CS370>"
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ make
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ sudo apt update && sudo apt install -y gcc build-essential libssl-dev
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gcc is already the newest version (4:13.2.0-7ubuntu1).
build-essential is already the newest version (12.10ubuntu1).
libssl-dev is already the newest version (3.0.13-0ubuntu3.9).
libssl-dev set to manually installed.
The following packages were automatically installed and are no longer required:
  libllvm19 libwayland-server0
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
```

Phase 1:

1A:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ nano object.c
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2:

2A:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ nano tree.c
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function ‘write_tree_level’:
tree.c:139:14: warning: implicit declaration of function ‘object_write’ [-Wimplicit-function-declaration]
  139 |     int rc = object_write(OBJ_TREE, data, len, id_out);
      |              ^~~~~~~~~~~~
tree.c:128:13: warning: ‘__builtin___strncpy_chk’ output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
  128 |             strncpy(te->name, dir_name, sizeof(te->name) - 1);
      |             ^
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
/usr/bin/ld: tree.o: in function `tree_from_index':
tree.c:(.text+0x713): undefined reference to `index_load'
collect2: error: ld returned 1 exit status
make: *** [Makefile:22: test_tree] Error 1
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./test_tree
-bash: ./test_tree: No such file or directory
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ gcc -Wall -Wextra -O2 -c index.c -o index.o
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ find .pes/objects -type f
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/24/48279dba995602855b8deca6a6a95a88eb2dd90006b776dc27512352505192
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/54/fb26dd5543206bf3bf304d788069994139d2a90b9a70319a700986970219bf
.pes/objects/cb/d5658a0cb00504678d8b8fcd559bdf3f9fe0042def5842c7aee311c7eadac9
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ xxd .pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2068  tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 0066 2246 63d2 3e6f  ello.txt.f"Fc.>o
00000020: 4d9d e9e2 c7e6 d876 4305 a92a 3830 a1a5  M......vC..*80..
00000030: 2d3d 5f4a a800 7b5c 39                   -=_J..{\9
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$
```

Phase 3:

3A:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ nano index.c
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ make pes
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ echo "hello" > file1.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ echo "world" > file2.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes add file1.txt file2.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  test_tree
  untracked:  commit.c
  untracked:  tree.h
  untracked:  commit.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  README.md
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  test_objects
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  index.c
```

3B:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776670939 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776670944 6 file2.txt
```

Phase 4:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ nano commit.c
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ nano commit.c
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ nano object.c
```

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ nano object.c
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ make pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ rm -rf .pes
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ echo "Hello" > hello.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes add hello.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes commit -m "Initial commit"
Committed: cbd5658a0cb0... Initial commit
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ echo "World" >> hello.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes add hello.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes commit -m "Add world"
Committed: 54fb26dd5543... Add world
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ echo "Goodbye" > bye.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes add bye.txt
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 2448279dba99... Add farewell
```

4A:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ ./pes log
commit 2448279dba995602855b8deca6a6a95a88eb2dd90006b776dc27512352505192
Author: Rekha Dhorigol <PES1UG24CS370>
Date:   1776671657

    Add farewell

commit 54fb26dd5543206bf3bf304d788069994139d2a90b9a70319a700986970219bf
Author: Rekha Dhorigol <PES1UG24CS370>
Date:   1776671587

    Add world

commit cbd5658a0cb00504678d8b8fcd559bdf3f9fe0042def5842c7aee311c7eadac9
Author: Rekha Dhorigol <PES1UG24CS370>
Date:   1776671563

    Initial commit

rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ cat .pes/refs/heads/main
2448279dba995602855b8deca6a6a95a88eb2dd90006b776dc27512352505192
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

4B:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/24/48279dba995602855b8deca6a6a95a88eb2dd90006b776dc27512352505192
.pes/objects/54/fb26dd5543206bf3bf304d788069994139d2a90b9a70319a700986970219bf
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/cb/d5658a0cb00504678d8b8fcd559bdf3f9fe0042def5842c7aee311c7eadac9
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ git add commit.c object.c
```

4C:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ cat .pes/refs/heads/main
2448279dba995602855b8deca6a6a95a88eb2dd90006b776dc27512352505192
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$
```

Final:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 690f708d9b80... Initial commit

Log after first commit:
commit 690f708d9b800c2af5ba04a4443bc9abf4f93b2ef90211f0255d0bb9b3f37017
Author: Rekha Dhorigol <PES1UG24CS370>
Date:   1776671781

    Initial commit


--- Second Commit ---
Committed: c553647a6c6a... Update file.txt

--- Third Commit ---
Committed: 4660ba1a56d1... Add farewell
```

```
commit c553647a6c6a593b9dbf860f73a6f381de4efd24536c17547877f2b5b453b0b7
Author: Rekha Dhorigol <PES1UG24CS370>
Date:   1776671781

    Update file.txt

commit 690f708d9b800c2af5ba04a4443bc9abf4f93b2ef90211f0255d0bb9b3f37017
Author: Rekha Dhorigol <PES1UG24CS370>
Date:   1776671781

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
4660ba1a56d14e7cff31a1f491f248d569fe75d4b1bba15dce2e5fff85e723d4

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/46/60ba1a56d14e7cff31a1f491f248d569fe75d4b1bba15dce2e5fff85e723d4
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/69/0f708d9b800c2af5ba04a4443bc9abf4f93b2ef90211f0255d0bb9b3f37017
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c5/53647a6c6a593b9dbf860f73a6f381de4efd24536c17547877f2b5b453b0b7
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

No. of commits:

```
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$ git log --oneline | wc -l
23
rdhor@Kulfi2:~/PES1UG24CS370-pes-vcs$
```